Stato dell'Arte nella Stima dei Costi di Progetti Software

Fabrizio Apuzzo Agostino Cesarano

August 3, 2026

1 Abstract

Il presente documento ha lo scopo di fornire una panoramica dello stato dell'arte nella stima dei costi di progetti software, esplorando e confrontando nel dettaglio due soluzioni. L'articolo [2] propone un approccio basato su Support Vector Machines (SVM) con un preprocessing basato su metodi algoritmici. L'articolo [1] propone un'architettura relativamente semplice e interpretabile, che si basa su una rete convoluzionale preceduta da una fase di clusterizzazione fuzzy. Le prestazioni di tale modello sono state superate solo da modelli di gran lunga più complessi che coinvolgono transformer e LLM, ottenendo risultati comunque comparabili. Questo approccio rappresenta un compromesso interessante tra accuratezza e complessità, rendendo l'esplorazione di tecniche simili un'area di ricerca promettente.

2 Introduzione

La stima dei costi di progetti software è un'attività critica per la pianificazione e la gestione dei progetti, il successo di un progetto dipende in larga misura dalla capacità di stimare accuratamente i costi e le risorse necessarie. Tuttavia, la natura imprevedibile e complessa del processo di sviluppo software rende questa attività particolarmente difficile. Negli ultimi decenni, sono stati sviluppati numerosi approcci per affrontare questa sfida, ciascuno con i propri punti di forza e limitazioni. In questo documento, esploreremo lo stato dell'arte nella stima dei costi di progetti software, concentrandoci su due approcci specifici: uno basato su Support Vector Machines (SVM) e l'altro su reti neurali convoluzionali (CNN) con clusterizzazione fuzzy.

3 Lo Stato dell'Arte

L'articolo [6] identifica e analizza quattro principali macro-approcci nello stato dell'arte per la stima dei costi di progetti software: stime basate su algoritmi,

stime basate sul giudizio di esperti, stime basate su modelli di apprendimento automatico, metodi ibridi.

3.1 Stime Basate su Algoritmi

I metodi basati su algoritmi sono approcci che utilizzano formule matematiche e modelli statistici per stimare lo sforzo in funzione di variabili come la dimensione e complessità del software da realizzare (spesso espressa in Function Point oppure Use Case Points) e le caratteristiche del team di sviluppo. A questa famiglia appartengono approcci come COCOMO e SLIM. Sebbene siano economici, veloci, prevedibili e facilmente interpretabili [3], l'efficacia di tali modelli è limitata dalla difficoltà di adattarsi a contesti diversi da quelli per cui sono stati progettati o tarati, con conseguente bisogno costante di aggiornarne i parametri o le definizioni stesse per riflettere le nuove tecnologie e metodologie di sviluppo. La letteratura [5] evidenzia cirticità nell'impiego del modello COCOMO II (già evoluzione di COCOMO I) a seguito della crescente diffusione di novità come i metodi Agile o i servizi web e cloud. Una nuova versione del modello, denominata COCOMO III, veniva infatti già discussa anni fa pur non essendo ancora stata ufficialmente rilasciata.

3.2 Stime Basate sul Giudizio di Esperti

Gli approcci basati sul giudizio di esperti si fondano sull'esperienza e la conoscenza di professionisti del settore per stimare i costi di progetti software. Questi metodi possono essere soggettivi e influenzati da bias cognitivi e ambiguità, ma la letteratura mostra come l'esperienza e la competenza degli esperti possano portare a stime accurate, il paper [4] evidenzia che 10 sui 16 studi analizzati mostrano un vantaggio in termini di accuratezza delle stime basate sul giudizio di esperti rispetto a quelle basate su metodi automatici. Alcuni dei metodi più comuni includono:

- **Planning Poker** - diversi esperti stimano in parallelo lo sforzo necessario per completare un'attività, confrontando poi le stime e discutendo eventuali discrepanze fino a raggiungere un consenso.
- **Analisi degli Story Point** - gli esperti stimano lo sforzo necessario per completare un'attività in termini di Story Point, che rappresentano una misura relativa della complessità e del tempo richiesto per completare l'attività.
- **Fuzzy Set** - gli esperti utilizzano insiemi fuzzy per rappresentare l'incertezza e la vaghezza nelle stime per ogni componente del progetto, consentendo una maggiore trasparenza nella valutazione dello sforzo richiesto.

Questi metodi non sono formalmente definiti come i modelli algoritmici, e la loro concreta applicazione può variare notevolmente a seconda del contesto e delle preferenze degli esperti coinvolti. Tuttavia, la loro flessibilità, la loro capacità di evolversi, e la loro capacità di cogliere informazioni contestuali uniche

e caratteristiche li rendono strumenti preziosi nella stima dei costi di progetti software.

3.3 Stime Basate su Modelli di Apprendimento Automatico

Gli approcci basati su machine learning sono potenzialmente più performanti rispetto ai metodi tradizionali, sono adattabili a contesti diversi e possono essere aggiornati con nuovi dati. Tuttavia, richiedono una quantità significativa di dati storici per addestrare i modelli e il loro processo decisionale non sono è facilmente interpretabile. Il costo operazionale di questi modelli può essere elevato, richiedendo risorse computazionali e competenze specialistiche per la loro implementazione e manutenzione. Gli approcci di machine learning più comuni includono:

- **Regressione Lineare** - un modello statistico che cerca di stabilire una relazione lineare tra le variabili indipendenti (ad esempio, la dimensione del software) e la variabile dipendente (ad esempio, lo sforzo stimato).
- **Alberi Decisionali** - un modello che utilizza una struttura ad albero per rappresentare le decisioni e le loro possibili conseguenze, basandosi su caratteristiche del progetto e del team di sviluppo.
- **Reti Neurali** - un modello ispirato al funzionamento del cervello umano, che utilizza strati di nodi interconnessi per apprendere relazioni complesse tra le variabili di input e output, come la dimensione del software e lo sforzo stimato.
- **Support Vector Machines (SVM)** - un modello che cerca di trovare un iperpiano ottimale per separare i dati in classi, basandosi su caratteristiche del progetto e del team di sviluppo.
- **Random Forest** - un modello che combina più alberi decisionali per migliorare la precisione delle stime, basandosi su caratteristiche del progetto e del team di sviluppo.
- **Gradient Boosting** - un modello che costruisce una serie di alberi decisionali in sequenza, dove ogni albero cerca di correggere gli errori del precedente, basandosi su caratteristiche del progetto e del team di sviluppo.

Tra quelli elencati, gli approcci basati su reti neurali sono la più grande macro-categoria di modelli di machine learning attualmente studiata e applicata nella stima dei costi di progetti software anche se la loro complessità e la difficoltà di interpretazione dei risultati possono rappresentare una barriera significativa per l'adozione in contesti industriali. Le sezioni successive approfondiscono le caratteristiche e le sfide associate ad alcuni di questi approcci in particolare.

3.4 Approcci Ibridi

Gli approcci ibridi combinano elementi dei metodi precedenti per sfruttare i punti di forza di ciascun approccio. Ad esempio, un approccio ibrido potrebbe utilizzare un modello algoritmico per fornire una stima iniziale, che viene poi raffinata attraverso il giudizio di esperti. Si distinguono due principali categorie di approcci ibridi:

- **Omogenei** - si combina lo stesso modello di base con diverse configurazioni
- **Eterogenei** - si combinano modelli sostanzialmente diversi

Di loro natura, gli approcci ibridi sono più robusti e generalmente più accurati rispetto ai metodi non ibridi, ma possono essere più complessi da implementare e richiedere più risorse.

4 SVM e Reti Neurali Convoluzionali a Confronto

...

References

[1] Mohammad Azzeh, Abedalrhman Alkhateeb, and Ali Nassif. Software effort estimation using convolutional neural network and fuzzy clustering. *Neural Computing and Applications*, 36:14449–14464, 05 2024.

[2] Sergio Di Martino, Filomena Ferrucci, Carmine Gravino, and Federica Sarro. Assessing the effectiveness of approximate functional sizing approaches for effort estimation. *Information and Software Technology*, 123:106308, 2020.

[3] Wei Lin Du, Danny Ho, and Luiz Fernando Capretz. A neuro-fuzzy model with SEER-SEM for software effort estimation. *CoRR*, abs/1508.00032, 2015.

[4] Magne Jorgensen and Martin Shepperd. A systematic review of software development cost estimation studies. *IEEE Transactions on Software Engineering*, 33(1):33–53, 2007.

[5] Tim Menzies, Ye Yang, George Mathew, Barry W. Boehm, and Jairus Hihn. Negative results for software effort estimation. *CoRR*, abs/1609.05563, 2016.

[6] Kevin James Tomescu, Niamh Gowran, Lorena Gomez, Eoin Delahunty, Andrew McCarren, Gerard Marks, Murat Yilmaz, Richard Messnarz, and Paul M. Clarke. A review of estimation in software engineering. In Murat Yilmaz, Paul Clarke, Andreas Riel, Richard Messnarz, Mikus Zelmenis, and Ivi Anna Buce, editors, *Systems, Software and Services Process Improvement*, pages 158–175, Cham, 2026. Springer Nature Switzerland.